Submission Worksheet

CLICK TO GRADE

https://learn.ethereallab.app/assignment/IT114-005-F2024/it114-milestone-4-chatroom-2024-m24/grade/bna24

Course: IT114-005-F2024
Assigment: [IT114] Milestone 4 Chatroom 2024 M24
Student: Bakr A. (bna24)

Submissions:

Submission Selection

1 Submission [submitted] 12/10/2024 6:54:01 AM

Instructions

COLLAPSE

- Implement the Milestone 4 features from the project's proposal document: https://docs.google.com/document/d/1ONmvEvel97GTFPGfVwwQC96xSsobbSbk56145XizQG4/view
- Make sure you add your ucid/date as code comments where code changes are done
- All code changes should reach the Milestone4 branch
- Create a pull request from Milestone4 to main and keep it open until you get the output PDF from this assignment.
- Gather the evidence of feature completion based on the below tasks.
- Once finished, get the output PDF and copy/move it to your repository folder on your local machine.
- Run the necessary git add, commit, and push steps to move it to GitHub
- Complete the pull request that was opened earlier
- Upload the same output PDF to Canvas

Branch name: Milestone4

Group

100%

Group: Features
Tasks: 4
Points: 9

COLLAPSE

Task

100%

Group: Features
Task #1: Client can export chat history of their current session (client-side)
Weight: ~0%
Points: ~0.01

COLLAPSE

Details:
For this requirement it's not valid to have another list keep track of messages. The goal is to utilize the location where messages are already present.

This must be a client-side implementation.

Columns: 1

Sub-Task

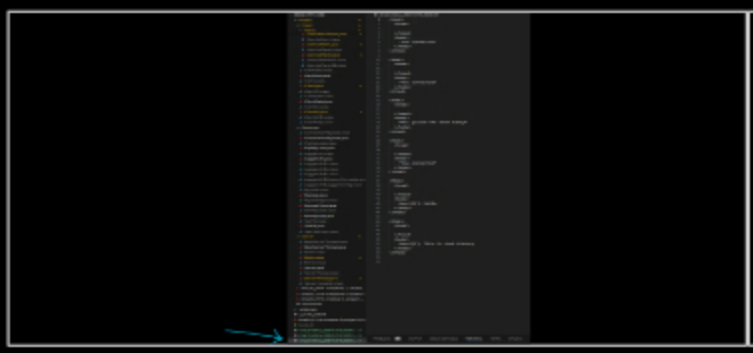
100%

Group: Features
Task #1: Client can export chat history of their current session (client-side)
Sub Task #1: Show a few examples of exported chat history (include the filename showing that there are multiple copies)

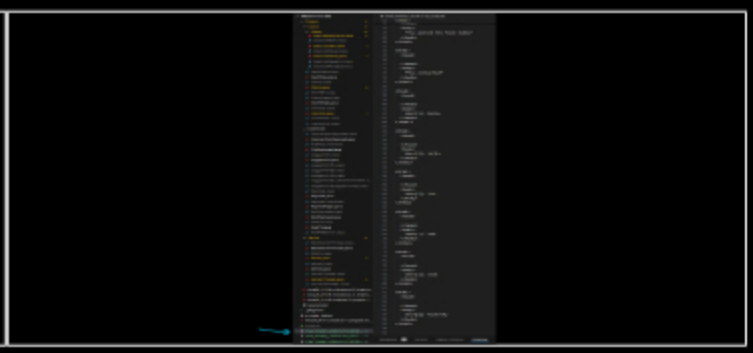
Task Screenshots

Gallery Style: 2 Columns

4 2 1



Export Chat #1



Export chat #2

Caption(s) (required) ✓
Caption Hint: *Describe/highlight what's being shown*

Sub-Task

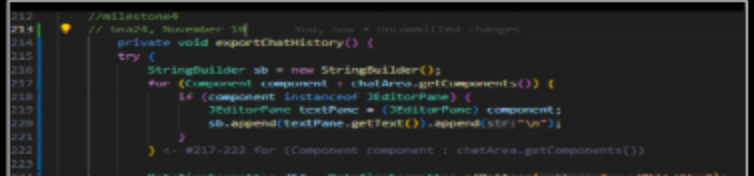
100%

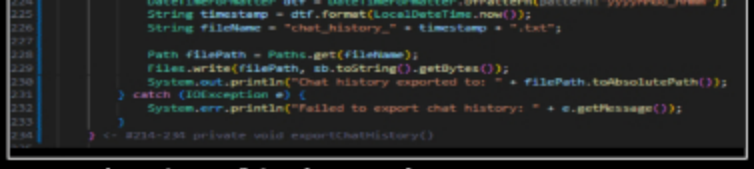
Group: Features
Task #1: Client can export chat history of their current session (client-side)
Sub Task #2: Show the code related to building the export data (where the messages are gathered from, the StringBuilder, and the file generation)

Task Screenshots

Gallery Style: 2 Columns

4 2 1





exportChatHistory() in chatpanel

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Explain in concise steps how this logically works

Response:

When the user press the "Export Chat" button, the code run through all message elements in the chat area and grabs each line of text. It collect these lines inside a StringBuilder. Then it create a formatted timestamp using year, month, day, hour, and minute, so the exported file has a readable name instead of random big numbers. After that, it writes the compiled text into a file and, if everything is fine, it print out where the file was saved otherwise, it shows a simple error message.

Sub-Task

100%

Group: Features

Task #1: Client can export chat history of their current session (client-side)

Sub Task #3: Show the UI interaction that will trigger an export

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Export Button

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Explain where you put it any why

Response:

A JButton labeled "Export Chat" is added to the input panel. When the user clicks this button, the actionListener is triggered. The actionListener then calls the exportChatHistory() method. This causes the chat data to be collected and saved to a file.

End of Task 1

Task

100%

Group: Features
Task #2: Client's Mute List will persist across sessions (server-side)
Weight: ~0%
Points: ~0.01

COLLAPSE

Details:
This must be a server-side implementation.

Screenshots of editors must have the frame title visible with your ucid and the client name.
Code screenshots must have ucid/data comments.

Columns: 1

Sub-Task

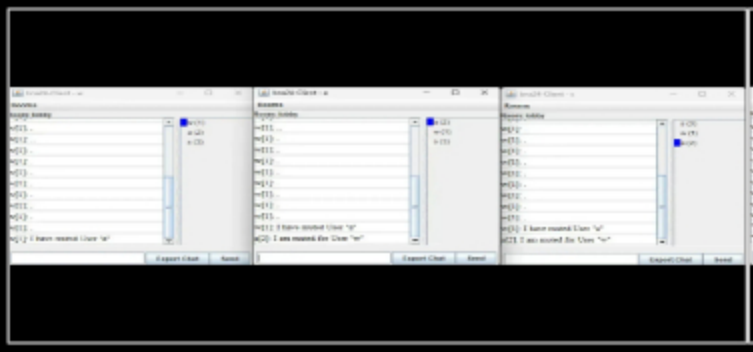
100%

Group: Features
Task #2: Client's Mute List will persist across sessions (server-side)
Sub Task #1: Show multiple examples of mutelist files and their content (their names should have/include the user's client name)

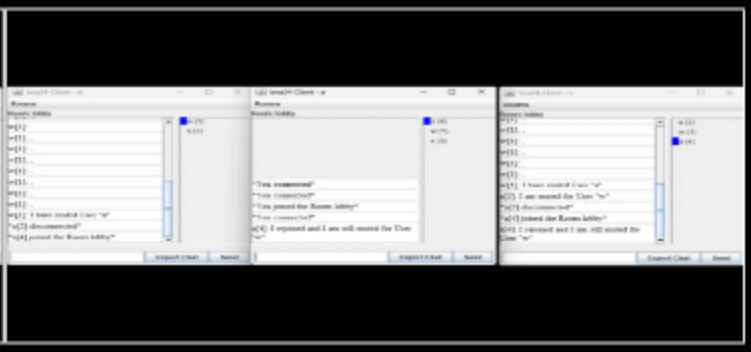
Task Screenshots

Gallery Style: 2 Columns

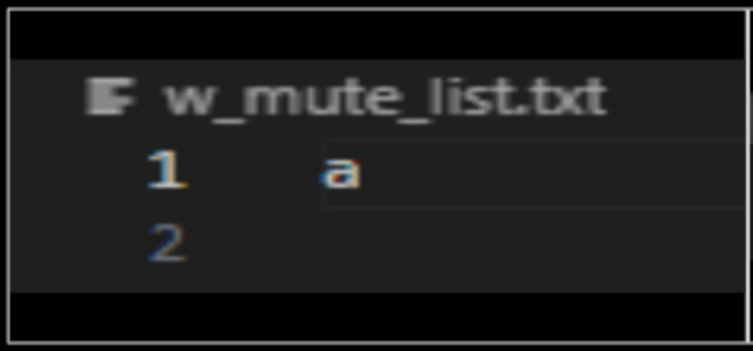
4 2 1



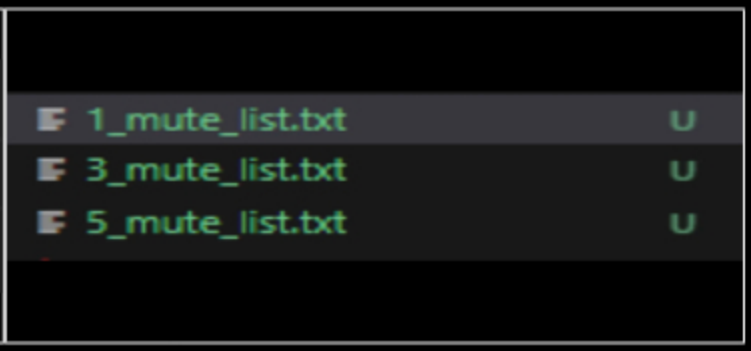
Muted User "a" without disconnecting



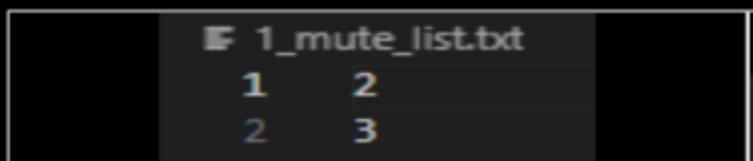
After user "a" disconnected and reconnected, he is still muted

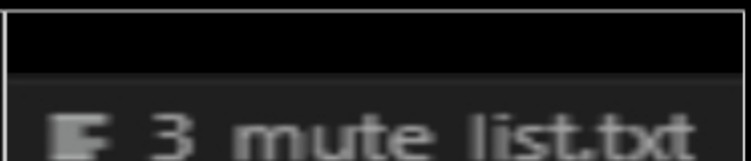


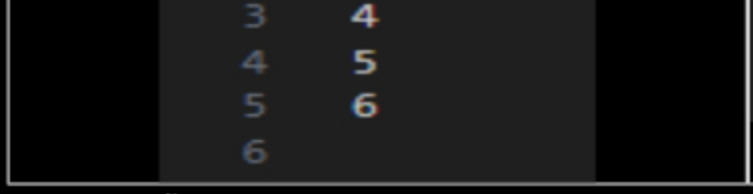
W_mute_list.txt



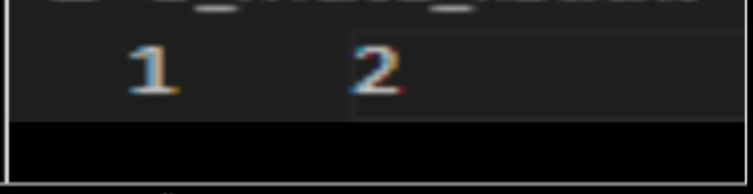
Users 1, 3, and 5 (those are the names I picked) mute files



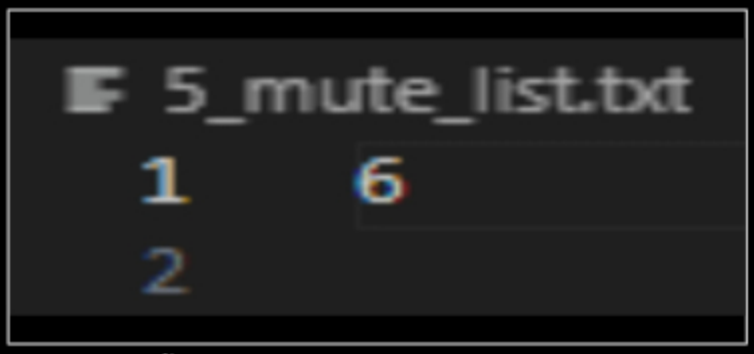




User 1 mute list



user 3 mute list



user 5 mute list

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

100%

Group: Features

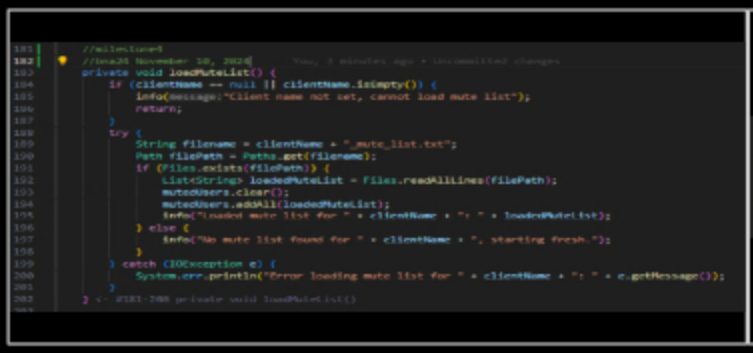
Task #2: Client's Mute List will persist across sessions (server-side)

Sub Task #2: Show the code related to loading the mutelist for a connecting client (and logic that handles if there's no file)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



```
//milestone4
//Ina24 November 10, 2024
private void loadMuteList() {
    if (clientName == null || clientName.isEmpty()) {
        Info(message:"Client name not set, cannot load mute list");
        return;
    }
    try {
        String filename = clientName + "_mute_list.txt";
        Path filePath = Paths.get(filename);
        if (Files.exists(filePath)) {
            List<String> loadedMuteList = Files.readAllLines(filePath);
            mutedUsers.clear();
            mutedUsers.addAll(loadedMuteList);
            Info("Loaded mute list for " + clientName + ": " + loadedMuteList);
        } else {
            Info("No mute list found for " + clientName + ", starting fresh.");
        }
    } catch (IOException e) {
        System.err.println("Error loading mute list for " + clientName + ": " + e.getMessage());
    }
} <- 8181-200 private void loadMuteList()
```

loadMuteList()

```
//Ina24
//November 27 2024
protected void processPayload(Payload payload) {
    try {
        switch (payload.getPayloadType()) {
            case CLIENT_CONNECT:
                ConnectionPayload connectionPayload = (ConnectionPayload) payload;
                setClientName(connectionPayload.getClientName());
                loadMuteList();
                System.out.println("Client connected with name: " + connectionPayload.getClientName());
                sendClientId(this.clientId);
```

calling loadMuteList

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Explain in concise steps how this logically works

Response:

When the client first connects, the server sets their chosen name and then tries to load their saved mute list. It look for a file named after that username, something like "bob_mute_list.txt". If the file is found, the server reads every line from it and puts those names into the client's mutedUsers set. This way, the client immediately continue with the same mute preferences from their last session. If no file is found, it just logs that fact and starts fresh with an empty list, meaning no one is muted yet.

Sub-Task

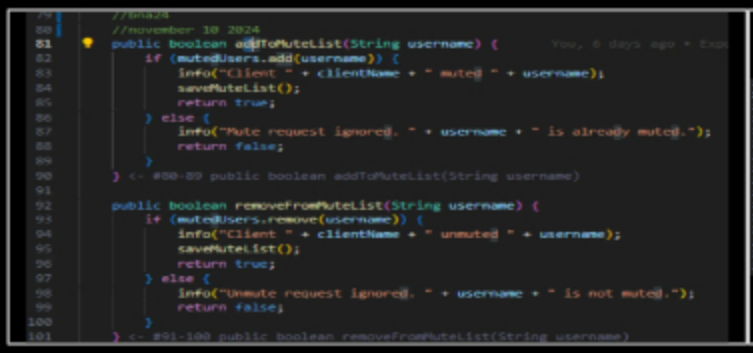
100%

Group: Features
Task #2: Client's Mute List will persist across sessions (server-side)
Sub Task #3: Show the code related to saving the mutelist whenever the list changes for a client

Task Screenshots

Gallery Style: 2 Columns

4 2 1



addToMuteList() & removeFromMuteList()

saveMuteList()

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Explain in concise steps how this logically works

Response:

When the client updates their mute preferences like muting a new user or unmuting someone previously muted. the server first modifies the internal mutedUsers set. Once that set is successfully changed, it calls a method to write the entire list of muted names out to a file named after the client's username. If the file doesn't exists, it create it. if it's already there, it overwrite the previous contents. Next time the client reconnects, the server read this file again, ensuring that the user's last known mute setting are kept. This means any changes made to who is muted or not contnues from one session to the next.

End of Task 2

Task

100%

Group: Features
Task #3: Clients will receive a message when they get muted/unmuted by another user
Weight: ~0%
Points: ~0.00

COLLAPSE

Details:

Screenshots of editors must have the frame title visible with your ucid and the client name.
Code screenshots must have ucid/data comments.

I.e., /mute Bob followed by a /mute Bob should only send one message because Bob can only be muted once until

Columns: 1

Sub-Task

100%

Group: Features
Task #3: Clients will receive a message when they get muted/unmuted by another user
Sub Task #1: Show the code that generates the well formatted message only when the mute state changes (see notes in the details above)

Task Screenshots

Gallery Style: 2 Columns

4 2 1

handeMute() & handleUnmute()

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Explain in concise steps how this logically works

Response:

When a client decide to mute or unmute another user, the server first updates the internal mute list. If the action actually change the state, like muting someone who wasnt muted before or unmuting someone who was, the server send a formatted message to both parties. The one who performed the action get a confirmation, something like "You have muted (user1)," while the target user gets a message that they were muted or unmuted by that specific user. If there's no actual change, for example trying to mute someone already muted, it just tell the sender that nothing new happened.

Sub-Task

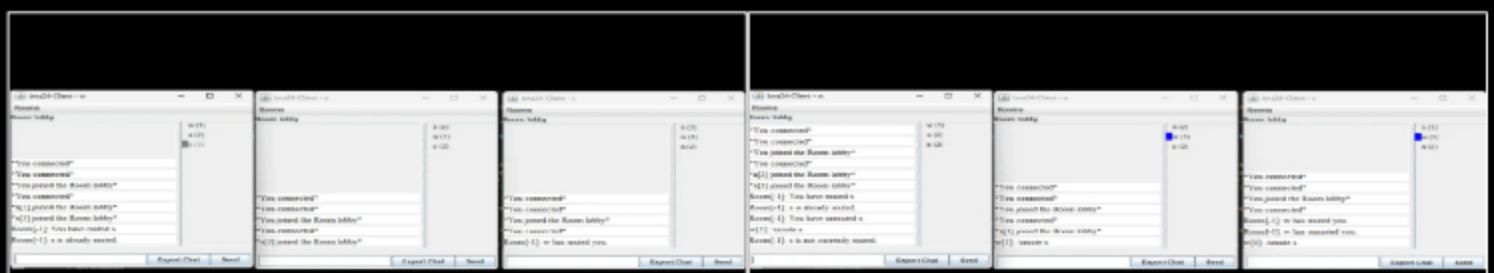
100%

Group: Features
Task #3: Clients will receive a message when they get muted/unmuted by another user
Sub Task #2: Show a few examples of this occurring and demonstrate that two mutes of the same user in a row generate only one message, do the same for unmute)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Mute example | Unmute Example

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 3

Task

100%

Group: Features
Task #4: The user list on the Client-side should update per the status of each user
Weight: ~0%
Points: ~0.01

COLLAPSE

Details:
Screenshots of editors must have the frame title visible with your ucid and the client name.
Code screenshots must have ucid/data comments.

Columns: 1

Sub-Task

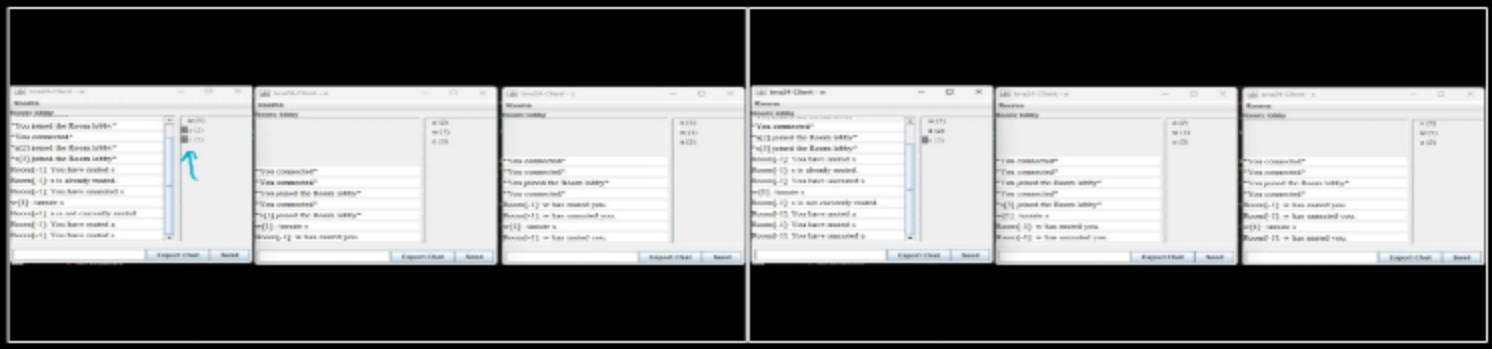
100%

Group: Features
Task #4: The user list on the Client-side should update per the status of each user
Sub Task #1: Show the UI for Muted users appear grayed out (or similar indication of your choosing) include a few examples showing it updates correctly when changing from mute/unmute and back

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Users "a" and "s" are grayed out because they are muted | User "a" is not longer gray because he has been unmuted

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

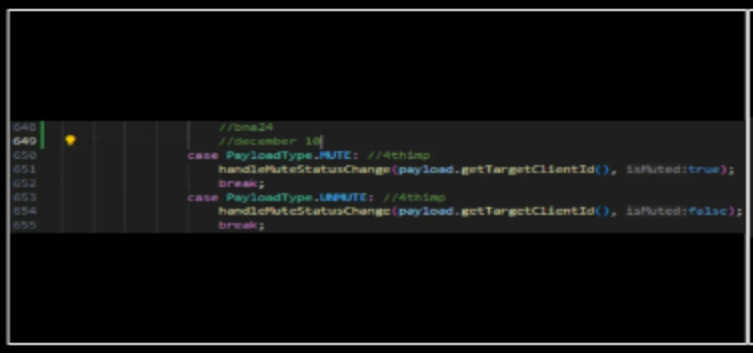
100%

Group: Features
Task #4: The user list on the Client-side should update per the status of each user
Sub Task #2: Show the code flow (client receiving -> UI) for Muted users appear grayed out (or similar indication of your choosing)

Task Screenshots

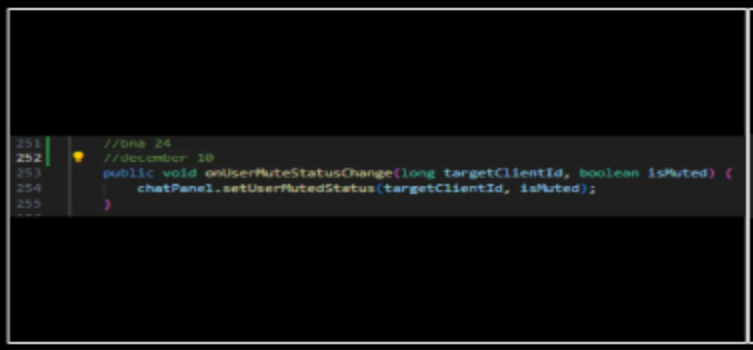
Gallery Style: 2 Columns

4 2 1

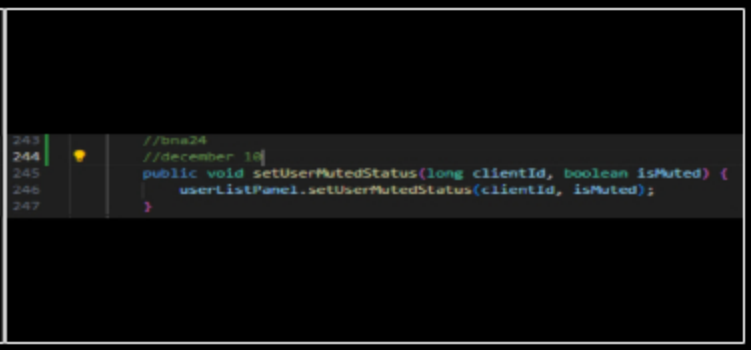


Client.java process payload

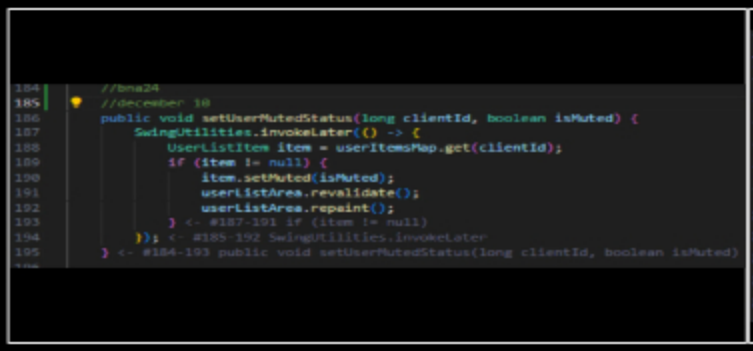
Cleint.java handlemuteStatusChange()



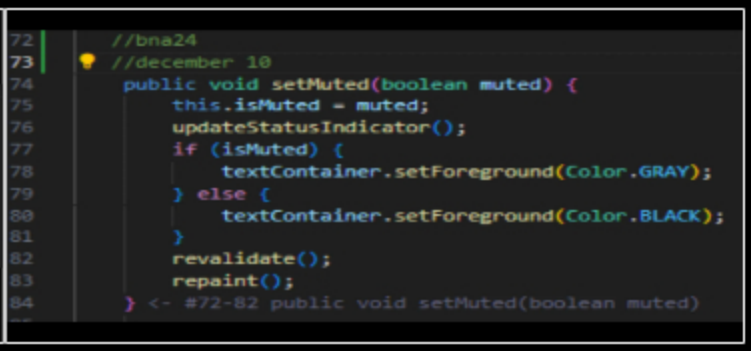
ClientUI.java onUserMuteStatusChange()



ChatPanel.java setUserMutedStatus()



UserListPanel.java setUserMutedStatus()



UserListItem.java setMuted()

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Explain in concise steps how this logically works

Response:

The server sends a MUTE or UNMUTE payload to the client, indicating a user's mute status has changed. The client's event handler receives this payload and calls a specific method (onUserMuteStatusChange) in ClientUI. ClientUI then tells ChatPanel to update that user's muted status. ChatPanel forwards this request to UserListPanel. UserListPanel finds the corresponding UserListItem and calls setMuted(isMuted) on it. UserListItem adjusts its display to reflect the new mute state.

Sub-Task

Group: Features

Task #4: The user list on the Client-side should update per the status of each user

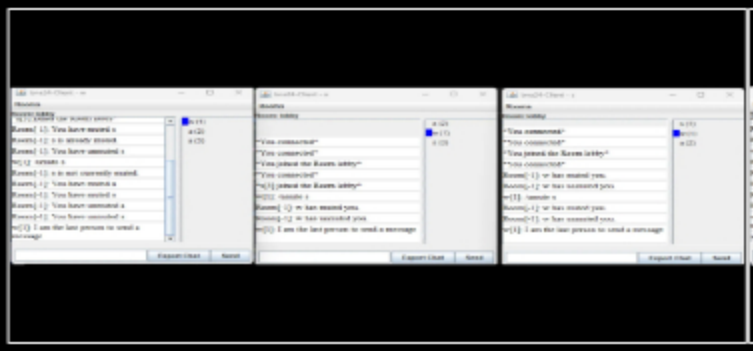
100%

Sub Task #3: Show the UI for Last person to send a message gets highlighted (or similar indication of your choosing)

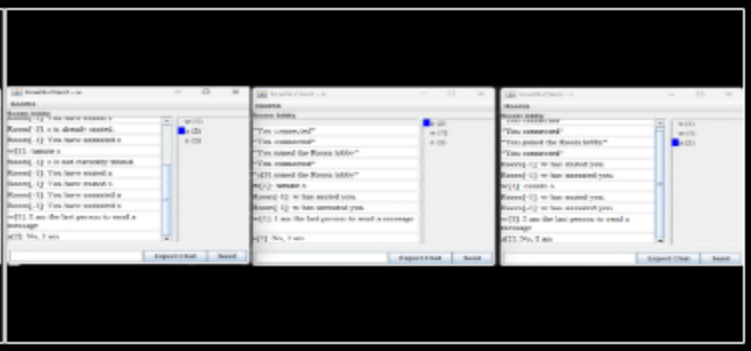
Task Screenshots

Gallery Style: 2 Columns

4 2 1



Last person example



Example Cont.

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

100%

Group: Features

Task #4: The user list on the Client-side should update per the status of each user

Sub Task #4: Show the code flow (client receiving -> UI) for Last person to send a message gets highlighted (or similar indication of your choosing)

Task Screenshots

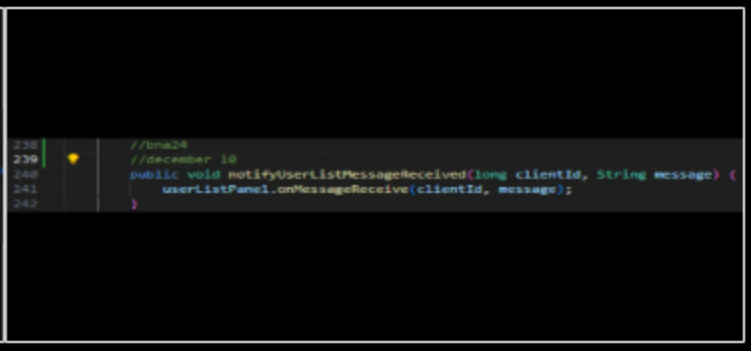
Gallery Style: 2 Columns

4 2 1

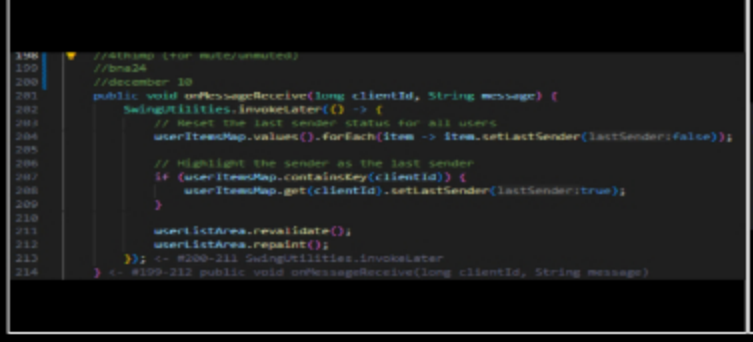
client.java processpayload()

processMessage(long clientId, String message) calls the message receive callback:

ClientUI.java onMessageReceive()



ChatPanel.java notifyUserListMessageReceived()

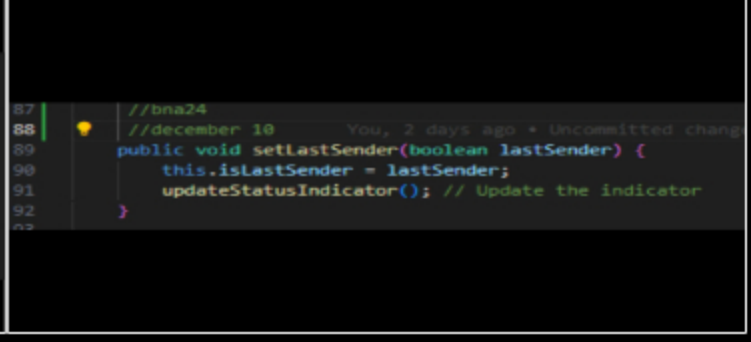


```
//bna24
//december 10
public void onMessageReceive(long clientId, String message) {
    SwingUtilities.invokeLater(() -> {
        // Reset the last sender status for all users
        userItemsMap.values().forEach(item -> item.setLastSender(lastSender:false));

        // Highlight the sender as the last sender
        if (userItemsMap.containsKey(clientId)) {
            userItemsMap.get(clientId).setLastSender(lastSender:true);
        }

        userListArea.revalidate();
        userListArea.repaint();
    }); <- #200-211 SwingUtilities.invokeLater
} <- #199-212 public void onMessageReceive(long clientId, String message)
```

UserListPanel.java onMessageReceive()



```
//bna24
//december 10        You, 2 days ago • Uncommitted chang
public void setLastSender(boolean lastSender) {
    this.isLastSender = lastSender;
    updateStatusIndicator(); // Update the indicator
}
```

UserListItem.java setLastSender()

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Explain in concise steps how this logically works

Response:

The server sends a MESSAGE payload to all clients whenever a user sends a message. The client receives the MESSAGE payload and calls its message event handler, eventually invoking onMessageReceive(clientId, message) in ClientUI. ClientUI updates the chat display and then calls chatPanel.notifyUserListMessageReceived(clientId, message) to inform the user list panel. ChatPanel calls userListPanel.onMessageReceive(clientId, message), passing along the ID of the sender. UserListPanel.onMessageReceive clears any existing "last sender" highlight and sets the current message sender as the new "last sender." The UserListItem for the sender updates its UI indicator to show that user as the most recent sender.

End of Task 4

End of Group: Features
Task Status: 4/4

Group

100%

Group: Misc
Tasks: 3
Points: 1

COLLAPSE

Task

100%

Group: Misc
Task #1: Add the pull request link for the branch
Weight: ~33%
Points: ~0.33

COLLAPSE

Details:

Note: the link should end with /pull/#

Task URLs

URL #1
https://github.com/BakrAbushaar/bna24-IT114-003

URL
https://github.com/BakrAbushaar/bna24-IT114-0

End of Task 1

Task

100%

Group: Misc
Task #2: Talk about any issues or learnings during this assignment
Weight: ~33%
Points: ~0.33

COLLAPSE

Task Response Prompt

Response:

I couldn't figure out how to do the call back for the fourth implementation and I asked for help from the professor. He guided me and told me to look at his projects and they helped me figure out how to do the call back for onMessageReceive()

End of Task 2

Task

100%

Group: Misc
Task #3: WakaTime Screenshot
Weight: ~33%
Points: ~0.33

COLLAPSE

Details:
Grab a snippet showing the approximate time involved that clearly shows your repository. The duration isn't considered for grading, but there should be some time involved

Task Screenshots

Gallery Style: 2 Columns

4 2 1

wakatime

End of Task 3

End of Group: Misc
Task Status: 3/3

End of Assignment